

Optimizing CNOT Gate Counts in Quantum Circuits via Look-Ahead: A Comparative Analysis of Heuristic vs. Naive Compilation Strategies

Johan Palm (qkv218)

Enrico Zanoni (swx519)

January 19, 2026

Supervisor: Prof. James Emil Avery

Abstract

Fault-tolerant quantum computing and NISQ applications both suffer from significant resource constraints, particularly regarding circuit depth and error rates. This report details the integration of QuCLEAR, an optimization algorithm proposed by Liu et al. designed to minimize quantum resources by maximizing the extraction of classical Clifford operations, into a Haskell-based compilation framework. The performance of the naive "Decompose-and-Push" approach is compared against the heuristic tree synthesis method. Results demonstrate that the integration achieves a substantial reduction in CNOT gate counts (over 40% for a circuit of 50 Pauli rotations on 50 qubits) compared to naive baselines. Furthermore, runtime analysis reveals a distinct scalability advantage: while the heuristic incurs a minor computational overhead for small circuits, it achieves faster compilation times for larger systems.

1 Introduction

Recent advances in quantum technology research have brought quantum computing significantly closer to achieving fault tolerance [9]. However, for fault-tolerant quantum algorithms, most notably Shor's algorithm that possess the theoretical capability to break RSA encryption, the practical resource requirements remain formidable. Current estimates suggest that factoring such keys would require approximately 8 hours of computation time utilizing 20 million noisy qubits [3].

Beyond the qubit overhead, energy consumption poses a critical scalability barrier, without optimization large-scale fault-tolerant systems could consume up to a gigawatt of power [2]. Significant resource constraints also define the current Noisy Intermediate-scale Quantum (NISQ) era. In this regime, the accumulation of noise is inextricably linked to circuit depth [8]. To mitigate these errors, the number of measurement "shots" must be drastically increased, driving up both runtime and financial cost. Consequently, for both near-term and future fault-tolerant architectures, minimizing circuit depth and optimizing gate sequences are crucial economic and technical necessities. With gate optimization for a given resource having an upper bound complexity of NQP [4], it is no trivial task.

To address these optimization challenges, researchers rely on efficient simulation techniques. The efficiency of Clifford circuit simulation is fundamentally rooted in the Gottesman-Knill theorem, which establishes that circuits composed of Clifford gates can be perfectly simulated on a classical computer in polynomial time [7].

Traditionally, this efficiency is demonstrated through the stabilizer formalism, which tracks the evolution of Pauli operators rather than the exponential full vector state.

1.1 Problem formulation

With quantum computation being a costly and error-prone resource, as much of the computation as possible should be executed on a classical computer [5]. A primary objective in quantum circuit optimization is to minimize the total number of gates and the entangling depth executed on noisy quantum hardware, as gate operations on current devices suffer from significantly higher error rates than their classical counterparts. This objective is shaped by the physical constraints of quantum processors, which typically support a limited universal gate set consisting only of arbitrary single-qubit rotations and a specific two-qubit entangling gate, most commonly the CNOT [6]. Consequently, high-level operations like multi-qubit Pauli rotations must be decomposed into ladders of these primitive gates, where the count of two-qubit gates becomes the dominant metric for error and cost. With Clifford gates being efficiently classically simulatable via the Gottesman-Knill theorem, the optimization goal is to transform a quantum simulation circuit U into a hybrid program. Isolating a final Clifford block U_{CL} such that an original unitary is preserved as $U = U_{CL}U'$ would allow for a classically simulatable stage and the quantum computation smaller as a result. This extraction is done by synthesizing the Pauli rotations to 1- and 2-qubit gates and commuting to the end of the circuit the classical

simulatable gates with the other unitaries.

The optimization problem can thus be formulated as: given a circuit containing a sequence of Pauli rotations, synthesize the Pauli rotations to 1- and 2-qubit gates and extract the Clifford circuit that minimizes the number of CNOT gates in the remaining quantum circuit U' .

This project focuses on the implementation and evaluation of the QuCLEAR algorithm [5] within a functional programming framework developed for the Advanced Topics in Programming Languages 2025/2026 course at the University of Copenhagen.

2 Theoretical Background

2.1 Simulatability of Circuits

The motivation lays in the fact that circuits using only Clifford gates can be efficiently simulated according to the Gottesman-Knill theorem. These are captured by the $\oplus L$ complexity class [7]. However, with the addition of non-Clifford gates, the complexity rises to BQP yielding the universal quantum gate set.

2.2 Pauli String Commutations

The crucial property allowing the extraction of Clifford gates is that the circuit for implementing an exponentiated Pauli string e^{-iPt} (i.e., a Pauli rotation circuit) weakly commutes with Clifford gates [1]. Let P_1 be a Pauli string and U_{CL} a Clifford circuit. The conjugated Pauli string P_2 is defined as:

$$P_2 = U_{CL}^\dagger P_1 U_{CL} \quad (1)$$

Consequently, the rotation operator satisfies the commutation relation:

$$e^{-iP_1 t} U_{CL} = U_{CL} e^{-iP_2 t} \quad (2)$$

Proof. Using the Euler expansion $e^{-i\theta P} = \cos \theta I - i \sin \theta P$ and the identity $I = U_{CL} U_{CL}^\dagger$:

$$\begin{aligned} e^{-iP_1 t} U_{CL} &= (\cos t I - i \sin t P_1) U_{CL} \\ &= U_{CL} \cos t - i \sin t P_1 U_{CL} \\ &= U_{CL} \left(\cos t I - i \sin t \underbrace{U_{CL}^\dagger P_1 U_{CL}}_{P_2} \right) \\ &= U_{CL} e^{-iP_2 t}. \quad \square \end{aligned}$$

A subset of these commutation rules is provided in Table 1.

Gate (U_{CL})	Input (P_1)	Output (P_2)
H	X	Z
H	Y	-Y
H	Z	X
$R_X(\frac{\pi}{2})$	X	X
$R_X(\frac{\pi}{2})$	Y	-Z
$R_X(\frac{\pi}{2})$	Z	Y
I	$P \in \{X, Y, Z\}$	P
CX	$X_C \otimes I_T$	$X_C \otimes X_T$
CX	$Z_C \otimes I_T$	$Z_C \otimes I_T$
CX	$I_C \otimes X_T$	$I_C \otimes X_T$
CX	$I_C \otimes Z_T$	$Z_C \otimes Z_T$

Table 1. Subset of commutation rules for common Clifford gates, where $P_2 = U_{CL}^\dagger P_1 U_{CL}$.

3 Clifford Extraction Methodology

The decomposition of an arbitrary multi-qubit Pauli rotation $R_P(\theta)$ is achieved through a standard synthesis protocol [6]. This procedure consists of five steps:

1. **Basis Diagonalization:** Single-qubit Clifford gates are applied to map the Pauli string P to the Z-basis (transforming $X \rightarrow Z$ and $Y \rightarrow Z$).
2. **Parity Computation:** A CNOT network (often structured as a chain or tree) computes the collective parity of the active qubits onto a single root qubit.
3. **Core Rotation:** The desired phase rotation $R_Z(\theta)$ is applied to the root qubit.
4. **Parity Uncomputation:** The inverse of the CNOT network is applied to disentangle the parity.
5. **Basis Restoration:** The inverse single-qubit Clifford gates are applied to return the system to the original computational basis.

3.1 Naive Extraction Strategy

To enable optimization via Clifford extraction, Pauli rotations must be decomposed into discrete components that allow Clifford gates to be commuted out. In the naive approach, the parity logic is constructed using a fixed linear topology.

For a Pauli string of weight w , this strictly requires $2(w-1)$ CNOT gates arranged in a generic ladder structure. Algorithm 1 details this decomposition, splitting a rotation into three parts: the pre-rotation Clifford block (G_{pre}), the core Z-rotation (G_{core}), and the post-rotation Clifford block (G_{post}). The definitions for the helper functions (e.g., ToZBasis, NaiveLinearLadder) are provided in Appendix A.

Algorithm 1: Naive Gadget Decomposition

Input: Rotation operator $Gate = R(P, \theta)$
Output: Tuple $(G_{post}, G_{core}, G_{pre})$

```

1 if  $P$  is Identity then
2   return  $(I, Gate, I)$ 
3 end
4  $G_{basis} \leftarrow \text{ToZBasis}(P)$ 
5  $G_{restore} \leftarrow G_{basis}^\dagger$ 
6  $Idxs \leftarrow \text{GetActiveIndices}(P)$ 
7 if  $|Idxs| < 2$  then
8    $L_{ladder} \leftarrow I; L_{unladder} \leftarrow I; t \leftarrow \text{Last}(Idxs)$ 
9 end
10 else
11    $(L_{ladder}, L_{unladder}) \leftarrow \text{NaiveLinearLadder}(Idxs)$ 
12 end
13  $G_{core} \leftarrow R_Z(\theta)$  on qubit  $t$ 
14  $G_{pre} \leftarrow L_{ladder} \cdot G_{basis}$ 
15  $G_{post} \leftarrow G_{restore} \cdot L_{unladder}$ 
16 return  $(G_{post}, G_{core}, G_{pre})$ 

```

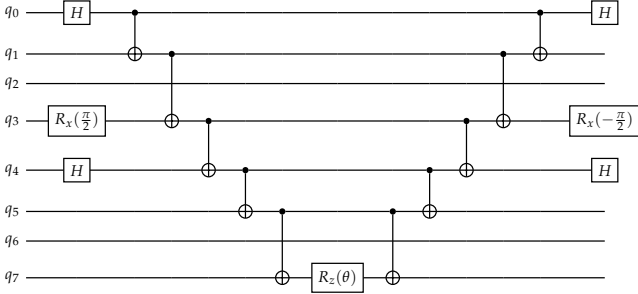


Figure 1. Naive decomposition of the Pauli rotation $e^{-i\frac{\theta}{2}(X_0Z_1I_2Y_3X_4Z_5I_6Z_7)}$.

3.2 Raw Circuit Expansion

The Raw Expansion strategy serves as the unoptimized baseline for compilation. It treats every Pauli rotation in the input circuit as an isolated operation. The compiler iterates strictly through the gate sequence, expanding every multi-qubit rotation into its full constituent gadget using the Naive Decomposition defined above.

While this method is computationally inexpensive in classical complexity, it produces quantum circuits with excessive depth. Specifically, because it blindly concatenates the full gadgets (Algorithm 2), it fails to eliminate redundant operations where the post-rotation block of one gate (G_{post}) could reduce the Hamming weight of the subsequent rotation axis.

3.3 Extraction via Naive Decompose-and-Push

The “Naive Decompose-and-Push” strategy (Algorithm 3) improves upon Raw Expansion by introducing a *Clifford Accumulator* (C_{acc}). This classical data structure tracks the aggregate effect of all the “uncompute” Clifford gates encountered so far, allowing the compiler to delay their physical execution.

Instead of resetting the quantum state after every rotation (as Raw Expansion does via G_{post}), this strategy effectively “pushes” the restoration operators into the

Algorithm 2: Raw Circuit Expansion

Input: List of input gates Ops_{in}
Output: Physical Circuit QC_{out}

```

1  $QC_{out} \leftarrow I$ 
2 for each gate in  $Ops_{in}$  do
3   // Decompose rotation into components
4    $(G_{post}, G_{core}, G_{pre}) \leftarrow \text{NaiveGadgetDecomposition}(gate)$ 
5    $QC_{out} \leftarrow QC_{out} \cdot G_{post} \cdot G_{core} \cdot G_{pre}$ 
6 end
7 return  $QC_{out}$ 

```

accumulator. When a new gate is encountered, it is first conjugated by the current accumulator to determine its effective Pauli string in the current reference frame:

$$Rot_{eff} = C_{acc}^\dagger \cdot Gate \cdot C_{acc} \quad (3)$$

The effective rotation is then decomposed using the Naive Gadget. Crucially, only the basis transformation and parity calculation (G_{pre}) and the core rotation (G_{core}) are appended to the physical circuit. The inverse restoration (G_{post}) is multiplied into C_{acc} . Importantly, the commutation of C_{acc} with the subsequent rotations might decrease or increase the Hamming weight of the Pauli string defining the axis, consequently decreasing or increasing the necessary number of entangling gates necessary to decompose them.

Algorithm 3: Decompose-and-Push Extraction

Input: List of input gates Ops_{in}
Output: Circuit $C_{acc} \cdot QC_{phys}$

```

1  $C_{acc} \leftarrow I; QC_{phys} \leftarrow I$ 
2 for each gate in  $Ops_{in}$  do
3   // 1. Map gate to current frame
4    $rot_{eff} \leftarrow C_{acc}^\dagger \cdot gate \cdot C_{acc}$ 
5   // 2. Decompose (See Alg. 1)
6    $(G_{post}, G_{core}, G_{pre}) \leftarrow \text{NaiveGadget}(rot_{eff})$ 
7   // 3. Absorb restoration into frame
8    $C_{acc} \leftarrow C_{acc} \cdot G_{post}$ 
9   // 4. Execute only prep and core
10   $Block \leftarrow G_{core} \cdot G_{pre}$ 
11   $QC_{phys} \leftarrow QC_{phys} \cdot Block$ 
12 end
13 // Returns the remaining frame + circuit
14 return  $C_{acc} \cdot QC_{phys}$ 

```

An example of decomposition using Algorithm 3 is in Figure 2. The original circuit is composed by two subsequent rotation around Pauli string (P_1): $Z_0Z_1Z_2Z_3$ and (P_2): $I_0Z_1Z_2I_3$. The full trace can be found in Appendix B.1.

3.4 Heuristic Extraction Strategy (QuCLEAR)

QuCLEAR adapts the approach in 3 by substituting Naive Gadget Decomposition with a more sophisticated greedy ‘look-ahead’ synthesis strategy. Instead of constructing an arbitrary CNOT tree for P_i , connections

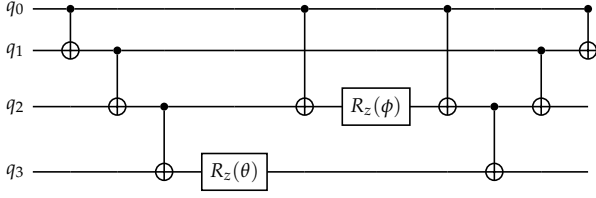


Figure 2. Extraction Algorithm with Naive Tree Synthesis ran on the circuit with Pauli strings $P_1 = Z_0Z_1Z_2Z_3$ & $P_2 = I_0Z_1Z_2I_3$.

are prioritized to specifically simplify the structure of the *next* operators $P_{i+1}, P_{i+2}, \dots$

The algorithm partitions qubits into four groups (S_I, S_X, S_Y, S_Z) based on the next operator and recursively synthesizes subtrees (Algorithm 4).

Algorithm 4: Recursive Smart Tree Synthesis

Input: Active indices Q_{active} , The ordered sequence of upcoming Paulis $Futures$
Output: Ladder L_{tree} , Inverse L_{inv} , Root q_{root}

```

1 if  $Futures$  is empty or  $|Q_{active}| \leq 1$  then
2   return NaiveLinearLadder( $Q_{active}$ )
3 end
4  $P_{next} \leftarrow \text{GetPauli}(Futures[0])$ 
5 Partition  $Q_{active}$  into  $S_I, S_X, S_Y, S_Z$  based on  $P_{next}$ 
6 Initialize empty list  $SubTrees$ 
7 for each non-empty set  $S \in \{S_I \dots S_Z\}$  do
8    $(L_{sub}, L_{inv\_sub}, r_{sub}) \leftarrow \text{Synth}(S, Futures[1 :])$ 
9   Add  $(Type(S), L_{sub}, L_{inv\_sub}, r_{sub})$  to  $SubTrees$ 
10 end
11  $Roots \leftarrow [(Type, r_{sub}) \text{ for each } SubTrees]$ 
12  $(L_{conn}, L_{inv\_conn}, q_{root}) \leftarrow \text{ConnectRoots}(Roots)$ 
13  $L_{tree} \leftarrow L_{conn} \cdot (\prod L_{sub}); L_{inv} \leftarrow (\prod L_{inv\_sub}) \cdot L_{inv\_conn}$ 
14 return  $(L_{tree}, L_{inv}, q_{root})$ 

```

Once the independent sub-trees are synthesized, their roots are interconnected by the heuristic `ConnectRoots` rather than a linear chain. This function prioritizes connections that reduce the Hamming weight of the subsequent Pauli string P_{next} . The connection logic proceeds in three stages:

1. **Z/Y Merge:** If both exist, the root of the S_Z tree is connected to the root of the S_Y tree, promoting the Y root.
2. **I/X Merge:** Similarly, the root of the S_I tree is connected to the root of the S_X tree, promoting the X root.
3. **Global Connection:** The survivor of the Z/Y branch is connected to the survivor of the I/X branch, establishing the final root for the parity calculation.

This hierarchical structure ensures that parity information flows in a specific order, maximizing the probability of reducing the complexity of future steps. An example

of decomposition using Algorithm 4 can be seen in the following Figure 3.

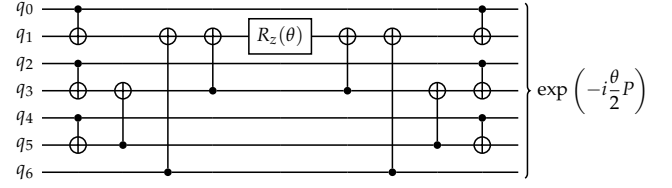


Figure 3. Decomposition of $e^{(-i\frac{\theta}{2}Z_0Z_1Z_2Z_3Z_4Z_5Z_6)}$ when the only next rotation is $e^{(-i\frac{\theta}{2}X_0X_1Y_2Y_3Z_4Z_5I_6)}$

The complete optimization process analogous to Algorithm 3, using Algorithm 4, is described in Algorithm 5.

Algorithm 5: Smart Clifford Extraction [5]

Input: List of Pauli rotations
 $Ops_{in} = [R(P_1, \theta_1), \dots, R(P_N, \theta_N)]$
Output: Physical circuit QC_{phys} , Extracted Clifford C_{extr}

```

1  $QC_{phys} \leftarrow I$ 
2  $C_{extr} \leftarrow I$ 
3 while  $Ops_{in}$  is not empty do
4   Pop current rotation  $R(P, \theta)$  from  $Ops_{in}$ 
5    $U_{basis} \leftarrow \text{ToZBasis}(P)$ 
6    $U_{restore} \leftarrow U_{basis}^\dagger$ 
7    $C_{extr} \leftarrow C_{extr} \cdot U_{restore}$ 
8   Update  $Ops_{in}$  by conjugating all ops with  $U_{basis}$ 
9    $Q_{active} \leftarrow \text{GetActiveQubits}(P)$ 
10   $(L_{tree}, L_{inv}, q_{root}) \leftarrow \text{SynthesizeTree}(Q_{active}, Ops_{in})$ 
11   $C_{extr} \leftarrow C_{extr} \cdot L_{inv}$ 
12  Update  $Ops_{in}$  by conjugating all ops with  $L_{tree}$ 
13   $G_{rot} \leftarrow R_Z(\theta)$  applied to  $q_{root}$ 
14   $Block \leftarrow G_{rot} \cdot L_{tree} \cdot U_{basis}$ 
15   $QC_{phys} \leftarrow QC_{phys} \cdot Block$ 
16 end
17 return  $QC_{phys}, C_{extr}$ 

```

As can be seen in Figure 4, Algorithm 4 produces a shallower circuit on the same example as in Figure 2. The full trace can be found in Appendix B.2.

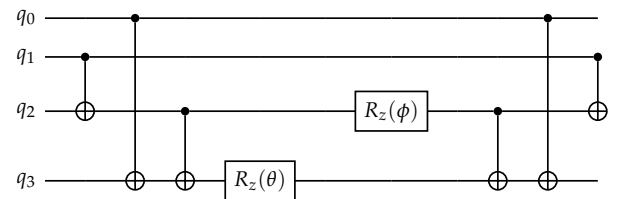


Figure 4. Heuristic Extraction ran on the circuit with Pauli strings $P_1 = Z_0Z_1Z_2Z_3$ & $P_2 = I_0Z_1Z_2I_3$.

4 Symplectic Implementation

The symplectic engine serves as the computational core of the optimizer, allowing to efficiently calculate the updated Pauli strings after weakly commuting with Clif-

ford gates. Instead of manipulating a high-dimensional complex state vector, the engine maintains the state as a boolean tableau of size $circuit\ depth \times (2n + 1)$, where n is the number of qubits. This tableau encodes the X and Z components of the Pauli operators that define the axis of the rotations, along with an additional column to track the sign of the rotation angle. The Haskell code for the symplectic representation is found in Listing 1.

```
1 data Symplectic = Symp {
2   xs  :: [Bool],
3   zs  :: [Bool],
4   sign :: Bool
5 } deriving (Show, Eq)
```

Listing 1. The Symplectic Data Structure

In Table 2 is the mathematical description using the above specified data structure: two vectors $xs, zs \in \mathbb{F}_2^n$ and a phase bit $sign \in \{0, 1\}$.

Gate	Vector Update (xs', zs')	Phase Update ($sign \oplus \dots$)
$CNOT_{c,t}$	$xs_t \leftarrow xs_t \oplus xs_c$ $zs_c \leftarrow zs_c \oplus zs_t$	$xs_c \wedge zs_t \wedge \neg(xs_t \oplus zs_c)$
H_i	$xs_i \leftarrow zs_i$ $zs_i \leftarrow xs_i$	$xs_i \wedge zs_i$
$R_x(\frac{\pi}{2})_i$	$xs_i \leftarrow xs_i \oplus zs_i$	$\neg xs_i \wedge zs_i$
$R_x(-\frac{\pi}{2})_i$	$xs_i \leftarrow xs_i \oplus zs_i$	$xs_i \wedge zs_i$

Table 2. Symplectic update rules for conjugation by Clifford gates.

5 Experimental Evaluation

To validate the efficacy of the optimization implementation, a benchmarking framework was developed to compare three baselines: (1) Raw Expansion (Algorithm 2), (2) Naive Optimization (Algorithm 3), and (3) Smart Optimization (Algorithm 5).

5.1 Experiment

All three baselines are ran on two experiments: 500 iterations of random rotations on 20 qubits with a gate depth of 20. Followed by, 100 iterations of random rotations on 50 qubits with a gate depth of 50. The baselines are evaluated by CNOT count and run time.

5.2 Experimental Setup

All benchmarks were executed on a local machine to ensure consistency.

- **Hardware:** MacBook Pro 2020 i5, 8GB RAM.
- **Software:** The implementation runs on GHC version 9.4.7.

Reproducibility: The complete source code and benchmark scripts are available at: github.com/enricozano/ATPL_finalproject

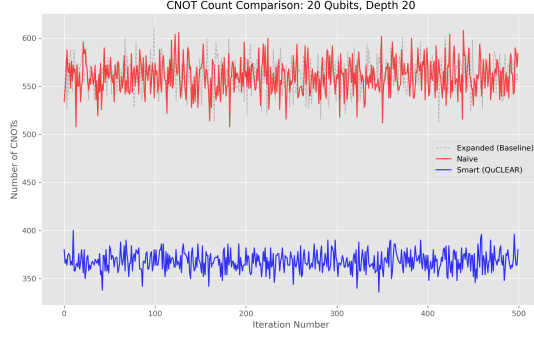
5.3 Results Analysis

CNOT Gate Reduction Analysis

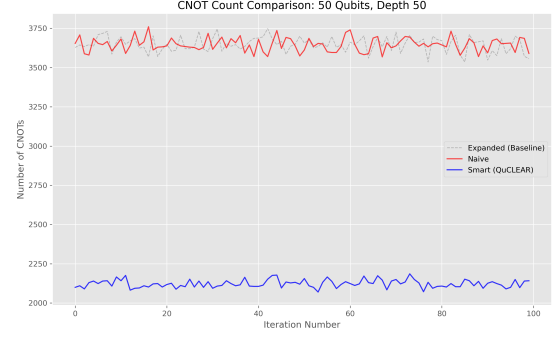
The results in Figures 5a and 5b demonstrate a significant divergence in resource scaling between the standard and heuristic synthesis methods. The Expanded baseline (dashed grey) and Naive strategy (red) show similar performance profiles, consistently requiring between 500 and 600 CNOT gates for the 20 qubits test and between 3550 and 3750 CNOT gates for the 50 qubits test. This highlights the limitations of disregarding the topology of the circuit; by enforcing a fixed linear connectivity structure ($0 \rightarrow 1 \rightarrow \dots \rightarrow n$), the Naive approach can not take advantage of the weak commutativity of the rotations with the Cliffords, having the same number of CNOT gates similar to the approach not even trying to commute gates. Therefore, it cannot effectively reduce the complexity of future Pauli rotations, resulting in a gate count that scales directly with the raw Hamming weight of the exponent of the operators. The high variance observed in the Naive trendline further suggests that its efficiency is highly sensitive to the random positioning between the Pauli strings and the fixed linear order. In contrast, the Heuristic strategy (blue) reduces the number of CNOT gates to approximately 300 for the 20 qubits test and 2100 for the 50 qubits one. This corresponds to a reduction of around 35% relative to the Naive baseline for the 20 qubits case and 41-42% for the 50 qubits case. This performance advantage validates the effectiveness of the greedy look-ahead mechanism employed by Algorithm 5. By dynamically partitioning qubits based on the structure of the axis of the subsequent rotation (grouping I,X,Y,Z terms), the heuristic constructs CNOT trees that not only satisfy the current parity requirements but actively minimize the number of qubits involved in future gates. The low variance in the blue trendline indicates a high degree of stability. Unlike the Naive approach, the heuristic method consistently identifies low cost circuit structures regardless of the specific randomness of the input instance, effectively mitigating the worst case penalties often associated with random circuit compilation. Moreover, the increased reduction in the 50 qubits test compared to the 20 qubits one suggests a positive dependency between the percentage reduction of the CNOTs and the complexity of the original quantum circuit.

Runtime Performance Analysis

The runtime performance data (Figure 6a & 6b) reveals a distinct trade-off between the computational overhead of the look-ahead mechanism and the downstream benefits of circuit simplification. In the 50-qubit experiment (Figure 6b), the Heuristic strategy demonstrates superior scalability in the tested cases. While the look-ahead mechanism introduces local complexity to the synthesis step, it actively simplifies the Pauli strings for subse-

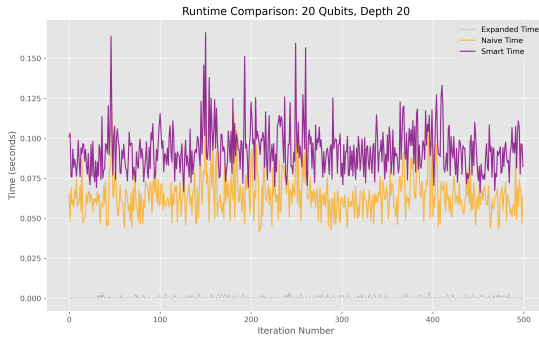


(a) 20 Qubits, Depth 20

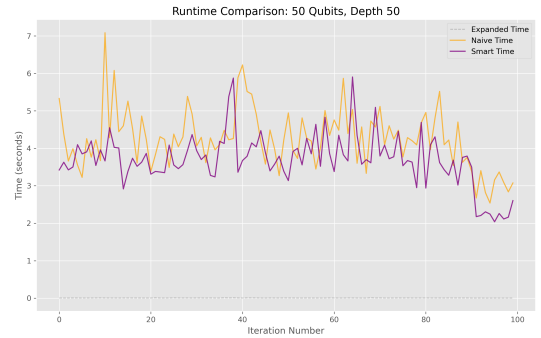


(b) 50 Qubits, Depth 50

Figure 5. Absolute CNOT count comparison. (a) 20 qubits showing consistent reduction. (b) 50 qubits showing over 40% reduction and significantly higher stability for the Heuristic strategy compared to the volatile Naive baseline.



(a) 20 Qubits, Depth 20



(b) 50 Qubits, Depth 50

Figure 6. Classical runtime comparison (s) over iteration. (a) 20 qubits showing overhead dominance where Naive is faster. (b) 50 qubits showing complexity normalization where Heuristic is faster.

quent rotations. As the compilation progresses, this “complexity normalization” ensures that the compiler handles lower-weight operators, preventing the massive runtime spikes observed in the Naive approach, which reach up to 7 seconds. Conversely, the 20-qubit experiment (Figure 6a) highlights the computational floor of the optimization strategy. Here, the Heuristic strategy is consistently slower than the Naive approach, with execution times spiking to approximately 0.15 seconds compared to the Naive baseline of roughly 0.10 seconds. In this regime, the problem space is small enough that the Naive linear decomposition is extremely fast. Although the Heuristic algorithm successfully simplifies the circuit structure, the computational time saved by synthesizing simpler gates is insufficient to compensate for the algorithmic overhead of the tree search. This suggests that while Algorithm 5 acts as a crucial stabilizer for large systems, it incurs a constant-factor overhead that dominates in small, shallow circuits.

6 Conclusion

This work presented the implementation of QuCLEAR, a heuristic-based Clifford extraction algorithm designed to optimize quantum circuits by reducing CNOT gate

counts. By employing a look-ahead strategy during CNOT tree synthesis, this strategy identifies symplectic redundancies that local gate decomposition methods miss. Experimental results confirm that the implementation consistently outperforms naive optimization strategies, achieving reductions of over 40% compared to unoptimized baselines on 50-qubit systems in circuit of 50 Pauli rotations.

Critically, the analysis highlights a trade-off in classical compilation resources. While the heuristic search introduces a constant-factor overhead that dominates execution time for small, shallow circuits, it proves beneficial for scalability. For larger systems, the algorithm effectively reduces the complexity of intermediate Pauli strings, resulting in faster compilation times compared to naive methods.

6.1 Limitations

While the experimental results validate the algorithmic approach, the current implementation is subject to specific constraints regarding efficiency, scope, and algorithmic fidelity.

First, regarding implementation efficiency, the current Haskell framework serves primarily as a proof-of-concept for functional quantum compilation. While

algorithmically correct, the codebase prioritizes semantic clarity over raw performance. A production-grade implementation would benefit from optimizing the code to mitigate the memory overhead associated with Haskell’s lazy evaluation. Second, the optimization scope is currently restricted to specific gate types. Moreover, regarding topological constraints, the synthesis engine currently assumes an ideal all-to-all qubit connectivity graph. This means the compiler generates CNOT gates between arbitrary qubit pairs without inserting the necessary SWAP chains required for execution on physical NISQ topologies. Finally, a notable algorithmic divergence from the original QuCLEAR proposal [5] is the absence of *commuting blocks*. The original algorithm pre-processes the input circuit by grouping adjacent Pauli strings into commuting clusters, allowing the compiler to reorder gates within these blocks to identify the optimal “next” operator for cancellation. Our implementation skips this pre-processing step and processes rotations strictly sequentially. As a result, it may miss optimization opportunities where a more compatible Pauli string is available slightly further down the circuit but separated by commuting operations.

6.2 Future Work

While this report analyzes runtime performance, a rigorous formal analysis of the algorithm’s space and time complexity remains necessary. Future studies should include precise memory profiling to quantify the allocation overhead of the recursive tree synthesis. Understanding the memory footprint is critical for determining the algorithm’s scalability to the fault-tolerant regime (hundreds or thousands of qubits), where the storage cost of symbolic Pauli manipulation could become a bottleneck.

A critical next step is to relax the assumption of all-to-all connectivity. For implementation on real-world NISQ devices, the CNOT tree synthesis must be adapted to respect physical coupling constraints. We propose two specific solutions to address this:

1. **Topology-Aware Tree Construction:** A new algorithm, conceptually similar to the current heuristic, could be developed to explicitly address topological constraints. This approach would integrate the insertion of SWAP gates, implemented via sequences of three CNOTs, directly into the tree synthesis process to bridge distant qubits.
2. **Software-Based Qubit Tracking:** To avoid the physical cost of SWAP gates, the compiler can implement a virtual mapping layer. By dynamically tracking the logical-to-physical qubit permutation in software, the algorithm can “virtualize” the movement of qubits, reducing the total circuit depth required for long-range interactions.

References

- [1] Timothée Goubault de Brugière and Simon Martiel. *Faster and shorter synthesis of Hamiltonian simulation circuits*. Apr. 4, 2024. DOI: [10.48550/arXiv.2404.03280](https://doi.org/10.48550/arXiv.2404.03280). arXiv: [2404.03280\[quant-ph\]](https://arxiv.org/abs/2404.03280). URL: <http://arxiv.org/abs/2404.03280>.
- [2] Marco Fellous-Asiani. *The resource cost of large scale quantum computing*. Feb. 13, 2022. DOI: [10.48550/arXiv.2112.04022](https://doi.org/10.48550/arXiv.2112.04022). arXiv: [2112.04022\[quant-ph\]](https://arxiv.org/abs/2112.04022). URL: <http://arxiv.org/abs/2112.04022>.
- [3] Craig Gidney and Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. In: *Quantum* 5 (Apr. 15, 2021), p. 433. ISSN: 2521-327X. DOI: [10.22331/q-2021-04-15-433](https://doi.org/10.22331/q-2021-04-15-433). arXiv: [1905.09749\[quant-ph\]](https://arxiv.org/abs/1905.09749). URL: <http://arxiv.org/abs/1905.09749>.
- [4] Adam Husted Kjelstrøm, Andreas Pavlogiannis, and Jaco van de Pol. *Exact Quantum Circuit Optimization is co-NQP-hard*. Oct. 18, 2025. DOI: [10.48550/arXiv.2510.16420](https://doi.org/10.48550/arXiv.2510.16420). arXiv: [2510.16420\[quant-ph\]](https://arxiv.org/abs/2510.16420). URL: <http://arxiv.org/abs/2510.16420>.
- [5] Ji Liu et al. *QuCLEAR: Clifford Extraction and Absorption for Quantum Circuit Optimization*. Feb. 22, 2025. DOI: [10.48550/arXiv.2408.13316](https://doi.org/10.48550/arXiv.2408.13316). arXiv: [2408.13316\[quant-ph\]](https://arxiv.org/abs/2408.13316). URL: <http://arxiv.org/abs/2408.13316>.
- [6] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Vol. 2010. Cambridge university press, 2010. URL: <https://michaelnielsen.org/qcqi/QINFO-book-nielsen-and-chuang-toc-and-chapter1-nov00-acro5.pdf>.
- [7] M. Van den Nest. *Classical simulation of quantum computation, the Gottesman-Knill theorem, and slightly beyond*. Oct. 14, 2009. DOI: [10.48550/arXiv.0811.0898](https://doi.org/10.48550/arXiv.0811.0898). arXiv: [0811.0898\[quant-ph\]](https://arxiv.org/abs/0811.0898). URL: <http://arxiv.org/abs/0811.0898>.
- [8] Ryuji Takagi, Hiroyasu Tajima, and Mile Gu. “Universal Sampling Lower Bounds for Quantum Error Mitigation”. In: *Physical Review Letters* 131.21 (Nov. 22, 2023), p. 210602. ISSN: 0031-9007, 1079-7114. DOI: [10.1103/PhysRevLett.131.210602](https://doi.org/10.1103/PhysRevLett.131.210602). URL: <https://link.aps.org/doi/10.1103/PhysRevLett.131.210602>.
- [9] Yukun Zhang et al. *Fault-tolerant quantum algorithms for quantum molecular systems: A survey*. Feb. 4, 2025. DOI: [10.48550/arXiv.2502.02139](https://doi.org/10.48550/arXiv.2502.02139). arXiv: [2502.02139\[quant-ph\]](https://arxiv.org/abs/2502.02139). URL: <http://arxiv.org/abs/2502.02139>.

A Glossary of Helper Functions

ToZBasis(P) Maps Pauli string P to the Z-basis (e.g., $X \rightarrow H, Y \rightarrow R_x(\frac{\pi}{2})$).

GetActiveIndices(P) Returns indices of non-identity operators in P .

Update Transforms future operations by conjugating them with gate C .

ConnectRoots($Roots$) Heuristic that links sub-tree roots to maximize Pauli cancellation in the *next* operator.

NaiveLinearLadder($Idxs$) Constructs a fixed linear CNOT chain ($q_i \rightarrow q_{i+1}$).

GetPauli($P[idx]$) Retrieves the single-qubit Pauli operator acting on a specific index idx within a larger Pauli string P .

B Comparative Algorithm Trace

This section contrasts the state evolution of the Naive and Heuristic strategies using the following sequence:

$$\text{Current } (P_1): Z_0 Z_1 Z_2 Z_3 \xrightarrow{\text{next}} \text{Future } (P_2): I_0 Z_1 Z_2 I_3$$

B.1 Trace 1: Naive Extraction (Decompose-and-Push)

The naive algorithm constructs a linear ladder for P_1 ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3$) without inspecting P_2 . The future operator P_2 is updated by conjugating it with the generated circuit (Basis Transform + Ladder).

Step-by-Step Evolution of P_2 :

$$\begin{aligned} \text{Initial } P_2 : & I_0 \otimes Z_1 \otimes Z_2 \otimes I_3 \\ \xrightarrow{\text{CNOT}(0,1)} & Z_0 \otimes Z_1 \otimes Z_2 \otimes I_3 \quad (I_0 \text{ propagates to } Z_0) \\ \xrightarrow{\text{CNOT}(1,2)} & Z_0 \otimes I_1 \otimes Z_2 \otimes I_3 \quad (Z_1 \text{ to } I_1) \\ \xrightarrow{\text{CNOT}(2,3)} & Z_0 \otimes I_1 \otimes Z_2 \otimes I_3 \quad (\text{does nothing}) \end{aligned}$$

Result: The transformed P'_2 has Hamming weight 2. No optimization was achieved; the subsequent rotation still acts on two qubits.

B.2 Trace 2: Heuristic Extraction (QuCLEAR)

The heuristic algorithm inspects P_2 to partition the active indices of P_1 into compatible sets: $S_Z = \{1, 2\}$ (indices where P_2 is Z) and $S_I = \{0, 3\}$ (indices where P_2 is I).

Synthesis Steps:

1. **Sub-trees:** Independent ladders are built for each set: CNOT(1, 2) and CNOT(0, 3).
2. **Connection:** The roots (q_2, q_3) are connected via CNOT(2, 3).

Step-by-Step Evolution of P_2 :

$$\begin{aligned} \text{Initial } P_2 : & I_0 \otimes Z_1 \otimes Z_2 \otimes I_3 \\ \xrightarrow{\text{CNOT}(1,2)} & I_0 \otimes I_1 \otimes Z_2 \otimes I_3 \quad (\text{cancels } Z_1) \\ \xrightarrow{\text{CNOT}(0,3)} & I_0 \otimes I_1 \otimes Z_2 \otimes I_3 \quad (\text{does nothing}) \\ \xrightarrow{\text{CNOT}(2,3)} & I_0 \otimes I_1 \otimes Z_2 \otimes I_3 \quad (\text{does nothing}) \end{aligned}$$

Result: The transformed P'_2 acts only on q_2 , reducing the Hamming weight to 1. Qubit q_1 is successfully idled.